https://miro.com/app/board/uXjVIByBwVI=/?share_link_id=231943841648

Deep Neural Networks Overfitting:
Regularization, Dropout, Early Stopping, Batch Normalization

Introduction

In this lecture, we move beyond shallow models to build deeper, more generalizable, and robust neural network.

You will learn about:

- The architecture of deep neural networks (DNNs)
- The problem of overfitting
- Regularization to improve generalization
- Dropout: reduce reliance on specific neurons
- Early stopping: when to stop training to avoid overfitting
- Batch normalization: Normalizes activations in intermediate layers to zero mean and unit variance

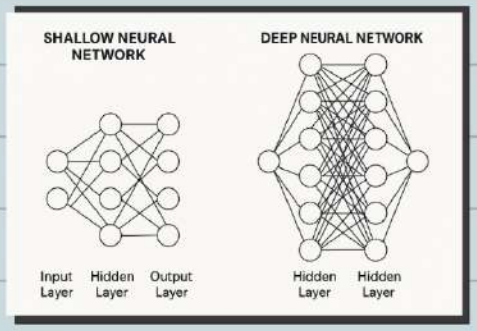
Deep Neural Networks (DNNs)

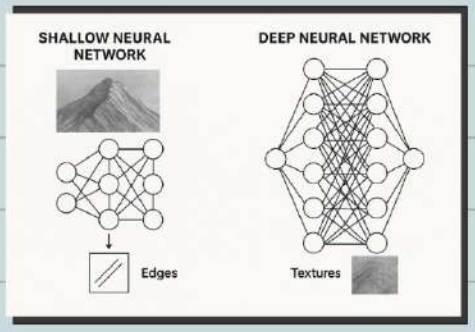
Why go deeper?

Deep neural networks:

- Are more parameter-efficient
- Capture hierarchical features (edges → textures → objects)
- Are more scalable for complex data like images

A DNN has multiple hidden layers



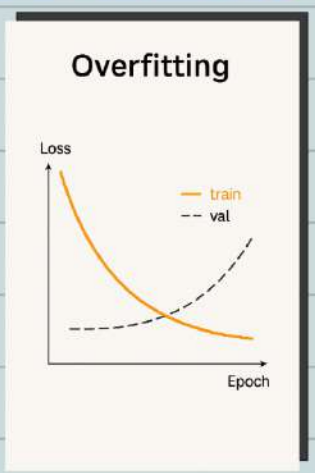


Each hidden layer learns more abstract features.

Overfitting

What is it?

1. Overfitting occurs when the model learns the training data too well, including noise and outliers.
2. It performs poorly on unseen (validation/test) data.

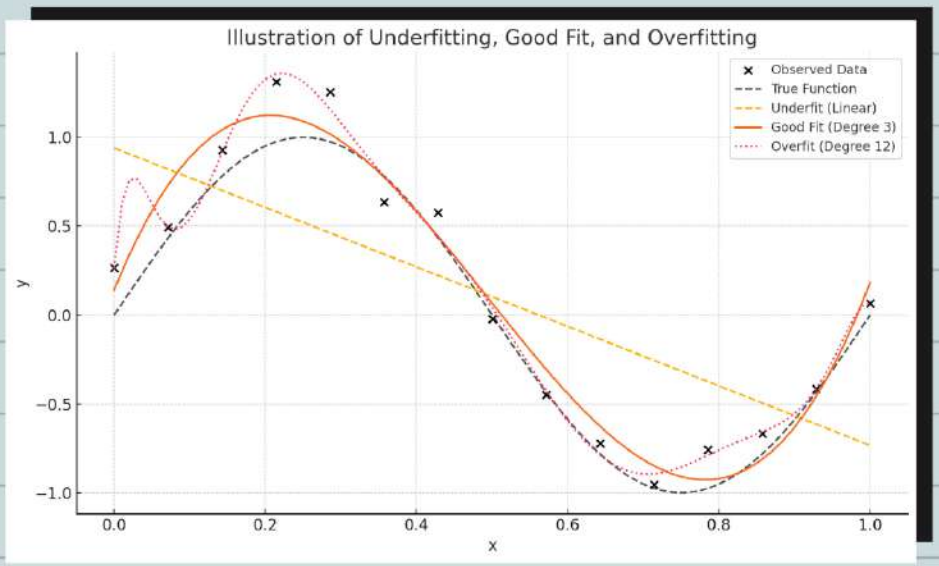


Analogy:
A student memorizing questions instead of understanding the concepts.

Why is overfitting bad:
- ✗ Poor generalization to new data
- ✗ Risk of incorrect predictions in real-world applications
- ✗ Wastes computational resources

Symptoms
- Training loss is low
- Validation loss starts increasing after some epochs

An example of overfitting

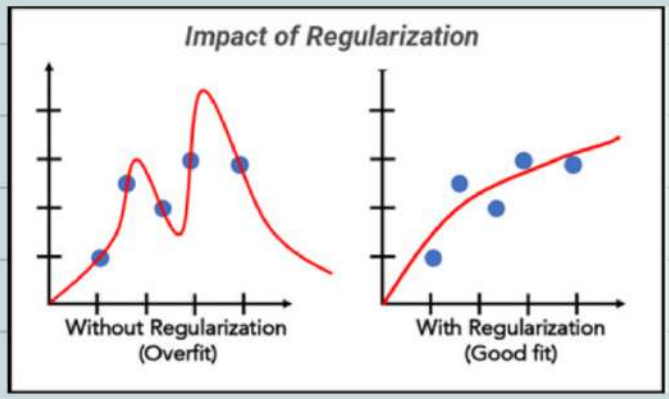


4 ways of preventing overfitting

There are many methods.
But we will focus on 4 methods in this lecture:

- ✓ Regularization
- ✓ Dropout
- ✓ Early stopping
- ✓ Batch normalization

Regularization



Theory:

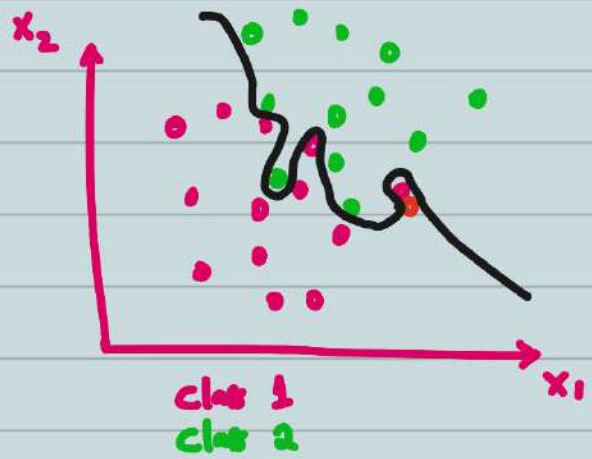
- Adds a penalty to large weight values.
- Common types:
  - L1 (Lasso): Adds absolute value of weights
  - L2 (Ridge): Adds squared value of weights

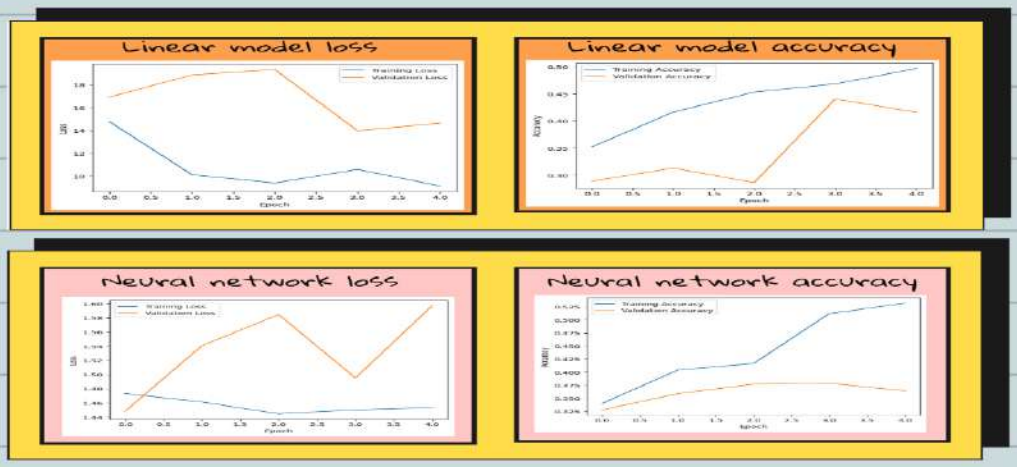
$L_2 \leftarrow \quad Loss = Loss + \lambda \sum_i (w)_i^2$

$L_1 \leftarrow \quad Loss = Loss + \lambda \sum_i |w_i|$

In regularization (like L1 or L2), we add a penalty term to the loss function that discourages large weights.

This is done because large weights make a model very sensitive to small changes in the input data, which often leads to overfitting.





Regularization implementation

Without Regularization

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
    keras.layers.Dense(config.hidden_nodes, activation="relu"),
    keras.layers.Dense(len(CLASS_NAMES), activation="softmax")
])
```

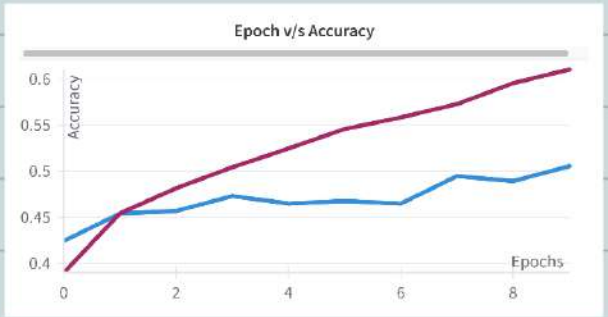
here, we using L2 Regularization so value of $L_1 = 0$ & L_2, λ value is 0.001

With regularization

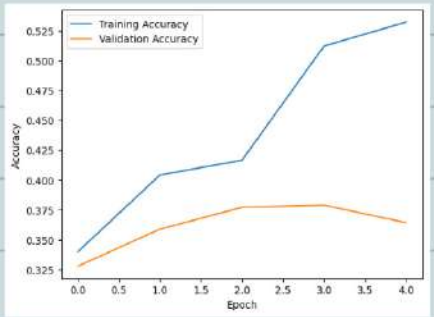
```
regularizer = tf.keras.regularizers.l1_l2(0, 0.001)

model = keras.Sequential([
    keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
    keras.layers.Dense(config.hidden_nodes, kernel_regularizer=regularizer, activation="relu"),
    keras.layers.Dense(len(CLASS_NAMES), kernel_regularizer=regularizer, activation="softmax")
])
```

Regularization results [5-flowers data]

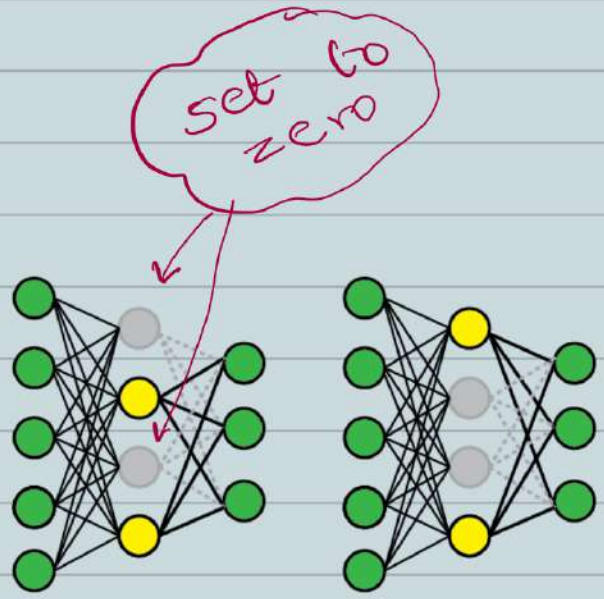


Without regularization [5-flowers data]



Dropout

Dropout is a regularization technique where, during training, we randomly "drop" (i.e., set to zero) a fraction of neurons in a layer at each forward pass.



That means:

- With probability p, a neuron is dropped (set to zero).
- With probability 1-p, it's kept and used normally.

This is done independently for each neuron and for each mini-batch during training.

Dropout is only applied during training. During inference, all neurons are used, but their outputs are scaled accordingly.

1. Each training batch sees a slightly different network.
2. During testing, no neurons are dropped, but their outputs are scaled by the dropout rate to maintain consistency.

What does this mean?

Suppose we apply a dropout rate of 0.5 to a layer. This means:

1. Each neuron has a 50% chance of being "dropped" (i.e., temporarily turned off - output set to 0).
2. Let's say a neuron survives. To maintain the expected total activation, its output is scaled up by 1/(1-0.5)=2.

So:

If a neuron is not dropped, it contributes more strongly during training to compensate for others being dropped.

Dropout implementation

Without dropout

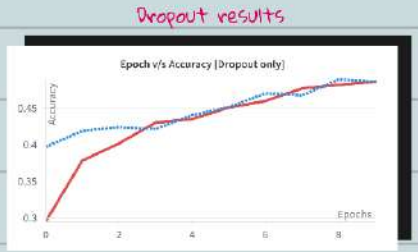
```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
    keras.layers.Dense(config.hidden_nodes, activation="relu"),
    keras.layers.Dense(len(CLASS_NAMES), activation="softmax")
])
```

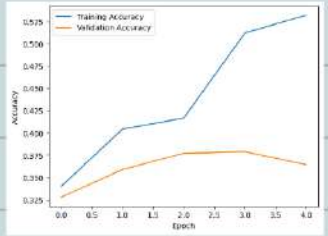
With dropout

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
    keras.layers.Dense(config.hidden_nodes, activation="relu"),
    keras.layers.Dropout(0.5),  # Drop 50% of the neurons during training
    keras.layers.Dense(len(CLASS_NAMES), activation="softmax")
])
```

only 1 line of code change for dropout.

Dropout results

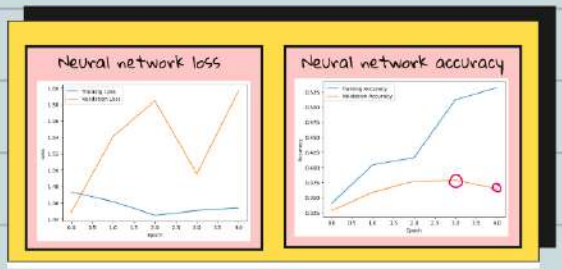




Early stopping

Concept:

1. Monitors validation loss or accuracy.
2. Stops training when the model starts to overfit (validation performance worsens).




```python
model.fit(
        train_dataset,
        validation_data=eval_dataset,
        epochs=config.epochs,
        callbacks=[
            WandbMetricsLogger(log_freq=5),
        ]
    )
```

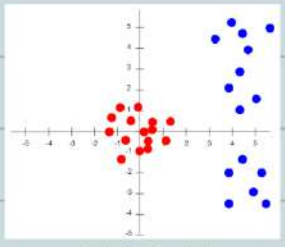
With early stopping

```python
model.fit(
        train_dataset,
        validation_data=eval_dataset,
        epochs=config.epochs,
        callbacks=[
            WandbMetricsLogger(log_freq=5),
            tf.keras.callbacks.EarlyStopping(monitor='val_loss',
                patience=2, restore_best_weights=True)
        ]
    )
```

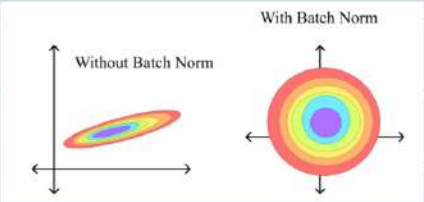
Batch normalization

```python
model = keras.Sequential([
        keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
        keras.layers.Dense(config.hidden_nodes, activation="relu"),
        keras.layers.Dense(len(CLASS_NAMES), activation="softmax")
    ])
```

Concept: Normalizes activations in intermediate layers to zero mean and unit variance.



- Original data
- Normalized data



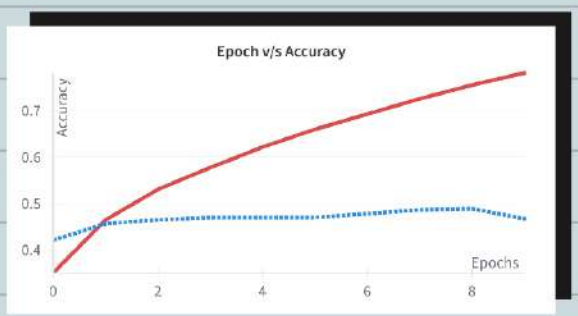
With batch normalization

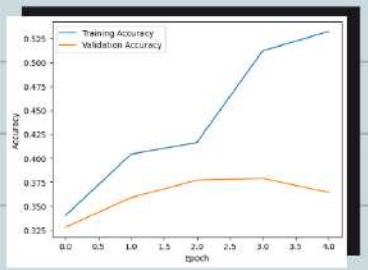
```python
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH,
        IMG_CHANNELS)),

    keras.layers.Dense(config.hidden_nodes),
    keras.layers.BatchNormalization(),
    keras.layers.Activation("relu"),

    keras.layers.Dense(len(CLASS_NAMES)),
    keras.layers.BatchNormalization(),
    keras.layers.Activation("softmax")
])
```

Batch normalization results





Apply all 4: Regularization, dropout, early stopping, batch normalization

```
model = keras.Sequential([
        keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS)),
        keras.layers.Dense(config.hidden_nodes, activation="relu"),
        keras.layers.Dense(len(CLASS_NAMES), activation="softmax")
    ])
```

BEFORE: Training + Validation

```
model.fit(
        train_dataset,
        validation_data=eval_dataset,
        epochs=config.epochs,
        callbacks=[
            WandbMetricsLogger(log_freq=5),
        ]
    )
```

AFTER: Layers

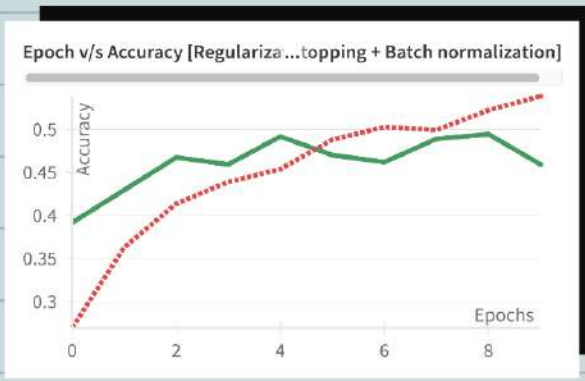
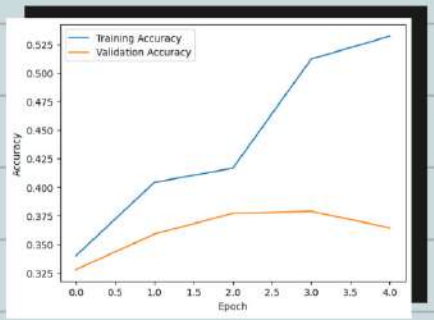
```
regularizer = tf.keras.regularizers.l1_l2(0, 0.1)

model = keras.Sequential([
        keras.layers.Flatten(input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS
            )),

        keras.layers.Dense(config.hidden_nodes, kernel_regularizer=regularizer
            ),
        keras.layers.BatchNormalization(),
        keras.layers.Activation("relu"),
        keras.layers.Dropout(0.5),

        keras.layers.Dense(len(CLASS_NAMES), kernel_regularizer=regularizer),
        keras.layers.BatchNormalization(),
        keras.layers.Activation("softmax")
    ])
```

AFTER: Training + Validation

```
model.fit( model.fit(
        train_dataset,
        validation_data=eval_dataset,
        epochs=config.epochs,
        callbacks=[
            WandbMetricsLogger(log_freq=5),
            tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3,
                restore_best_weights=True)
        ]
    ))
```

Results after implementing regularization + dropout + early stopping + batch normalization